Final_Project Check In

Greg Fisher

2026-09-29

1) Download the Sample RNA-seq Count Matrix and associated Metadata.

```
# Read in both data sets
count.df <- read.csv(file = "data/counts.csv", header = TRUE)
meta_data.df <- read.csv(file = "data/meta_data.csv", header = TRUE)
```

Data wrangling over the next sequence of steps

```
# Use only 5 columns of data in the meta_data dataframe to make processing faster
metadata.modified <- meta_data.df %>%
  dplyr::select(1,10,23,29,41) %>%
  # The barcode variable will eventually be used to join the metadata to the count data
  # The hyphen in the barcode variable needs to be changed to a dot
  mutate(barcode = str_replace_all(barcode, "-", "."))
```

```
# Convert count.df to long format
# This is required for many of the ggplot graphs and many other functions in R
# Use the pivot_longer function to convert the count dataframe to one I am calling
# dat.long which will end up containing three columns - the Ensembl ID, one called
# Samples which will contain all of the individual sample values and one called
# Counts which will contain the gene_expression count values

dat.long <- count.df %>%
  pivot_longer(
    cols = -X,         # Pivot all columns except the one containing the Ensemble ID
    names_to = "Samples",     # Column name for your old wide headers
    values_to = "Counts"     # Column name for your data values
  )
```

```
#
dat.long.enriched <- dat.long %>%
  mutate(X = sub("\\.[^.]*$", "", X)) %>%
  inner_join(., metadata.modified, by = c("Samples" = "barcode"))
```

```
# Map the Ensembl IDs to their actual gene symbol

library(AnnotationDbi)
library(org.Hs.eg.db)

count.df.mod <- count.df %>%
  mutate(X = sub("\\.[^.]*$", "", X))

ensemble.ids <- count.df.mod %>% pull(X)
```

```r
ensembl_ids <- ensemble.ids

# Map IDs to Gene Symbols
gene_symbols <- mapIds(
  x = org.Hs.eg.db,
  keys = ensembl_ids,
  keytype = "ENSEMBL",
  column = "SYMBOL",
  multiVals = "first" # Keeps the first match if duplicate mappings occur
)

df <- data.frame(names(gene_symbols),gene_symbols)
```

```r
dat.long.genes <- dat.long.enriched %>%
#  mutate(X = sub("\\.[^.]*$", "", X)) %>%
  inner_join(., df, by = c("X" = "names.gene_symbols."))
```

All of the data wrangling has been completed up to this point. Additional steps will be required when we create the heat map

2) Gene selection and Summary Statistics

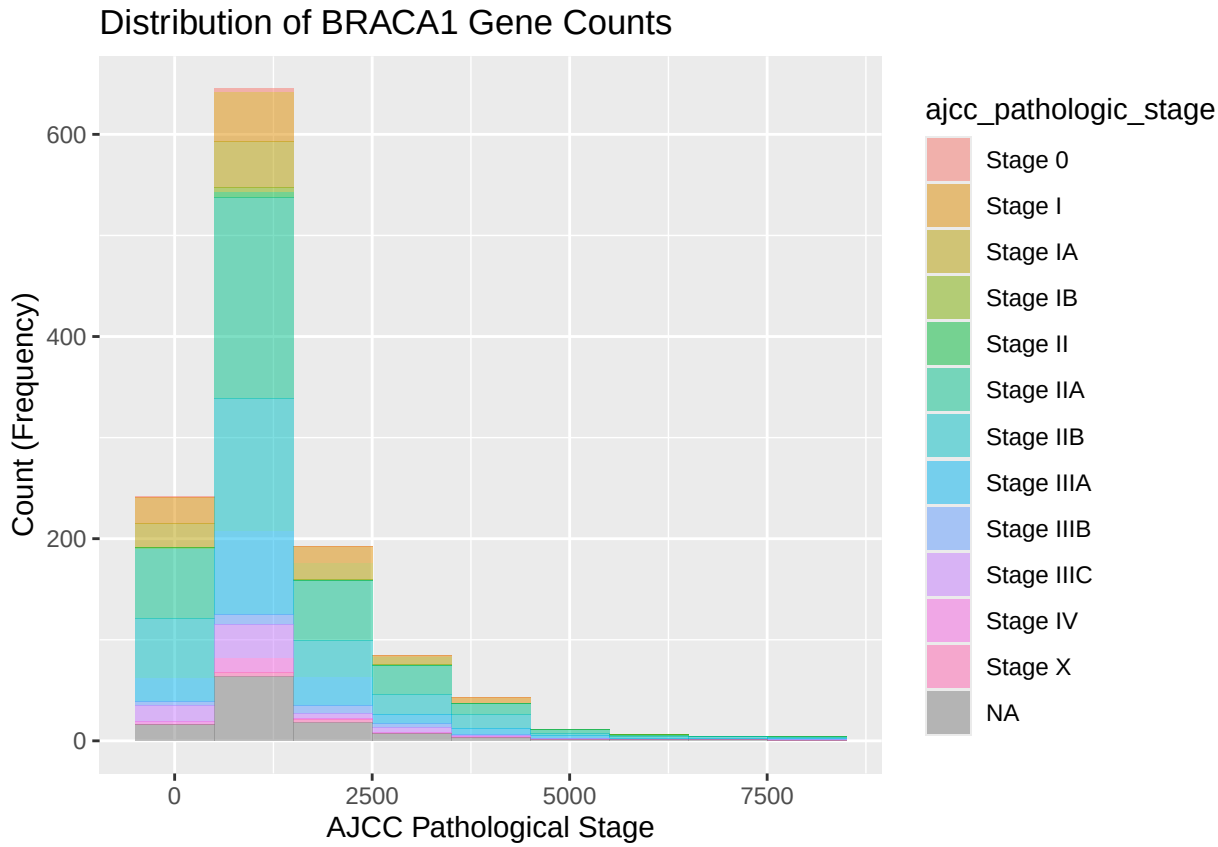
```
summary_stats <- dat.long.genes %>%
  filter(gene_symbols == 'BRCA2') %>%
  summarize(
    min = min(Counts, na.rm = TRUE),
    max = max(Counts, na.rm = TRUE),
    mean = mean(Counts, na.rm = TRUE),
    median = median(Counts, na.rm = TRUE),
    sd = sd(Counts, na.rm = TRUE)
  )
summary_stats
```

```
## # A tibble: 1 x 5
##     min   max  mean median    sd
##   <int> <int> <dbl>  <int> <dbl>
## 1    17  7206  749.    529  717.
```

3) Visualization

```
# a) Create a Histogram: Use ggplot2 to generate a histogram of the count data for the selected gene.
# This visualization should effectively display the distribution of the counts.

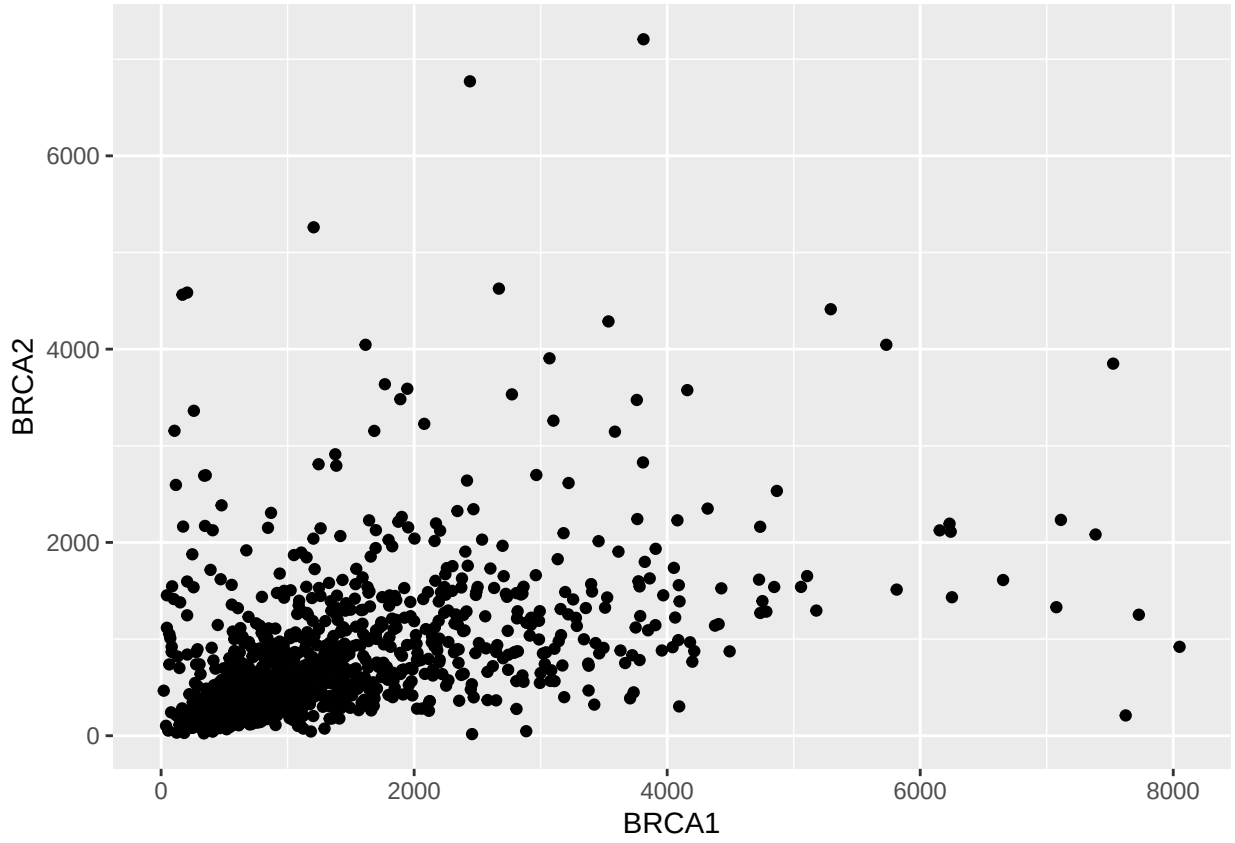
dat.long.genes %>%
  filter(gene_symbols == 'BRCA1') %>%
  ggplot(., aes(x=Counts, fill = ajcc_pathologic_stage)) +
  geom_histogram(alpha = 0.5, binwidth = 1000) +
  labs(
    title = "Distribution of BRACA1 Gene Counts",
    x = "AJCC Pathological Stage",
    y = "Count (Frequency)"
  )
```



```r
# b) Create a Scatter Plot: Select a second gene from the dataset. Create a scatter plot using
# ggplot2 to compare the count data of the two selected genes.

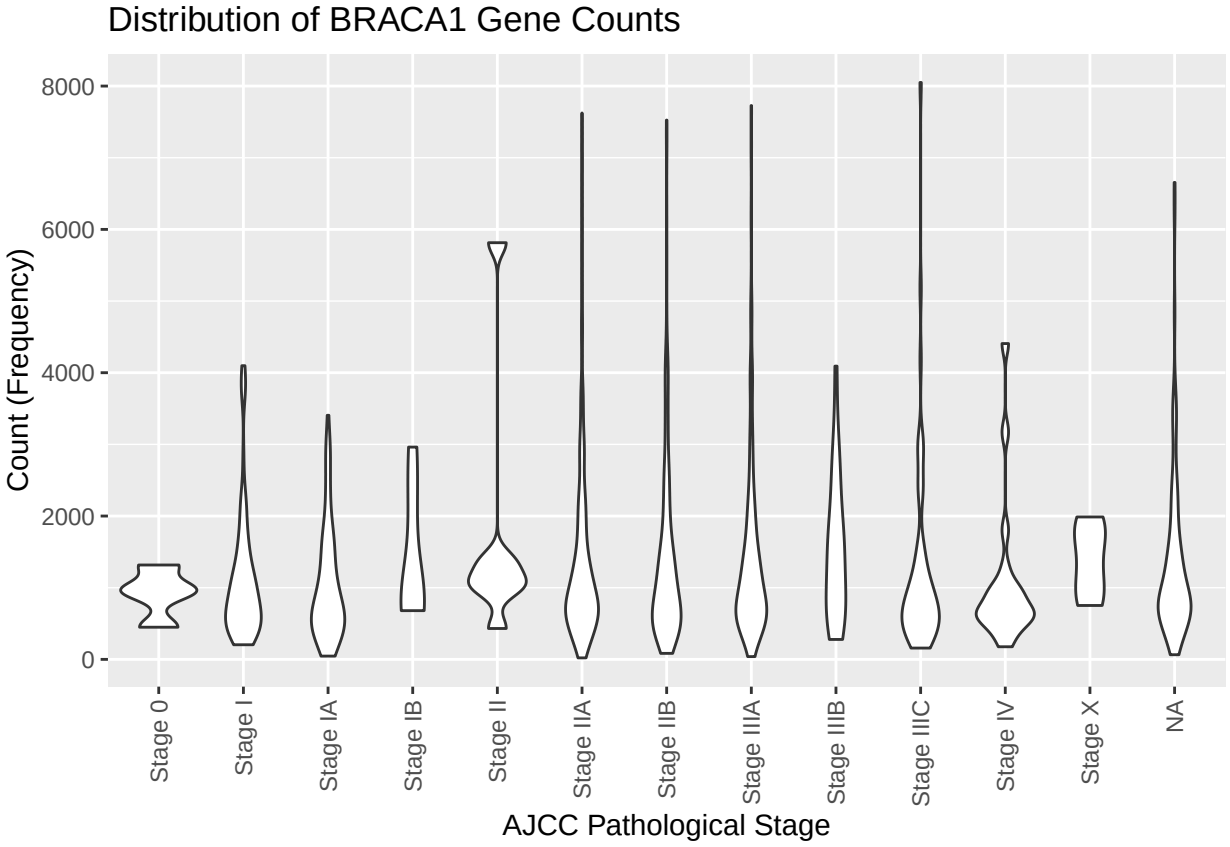
genes.of.interest <- c('BRCA1', 'BRCA2')

#df_test <- dat.long.genes %>%
dat.long.genes %>%
#  filter(gene_symbols == 'BRCA1' | gene_symbols == 'MYCBP2') %>%
  filter(gene_symbols %in% genes.of.interest) %>%
  dplyr::select(-1) %>%
  pivot_wider(names_from = gene_symbols, values_from = Counts) %>%
  ggplot(., aes(x=BRCA1,y=BRCA2)) +
  geom_point()
```



```r
# c) Create a Violin Plot: Select one covariate from your metadata. Using the count data from the first
   # gene and the selected covariate, generate a violin plot that illustrates the distribution of count
   # data based on the covariate. For example, if you choose "primary_diagnosis", your plot should disp

dat.long.genes %>%
  filter(gene_symbols == 'BRCA1') %>%
  ggplot(., aes(x=ajcc_pathologic_stage,y=Counts)) +
  geom_violin() +
  labs(
    title = "Distribution of BRACA1 Gene Counts",
    x = "AJCC Pathological Stage",
    y = "Count (Frequency)"
  ) +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))
```



```r
# d)    Heatmap Analysis:
# i)    Select 10 genes: Choose a set of 10 different genes from the count matrix for your heatmap.
# ii)   Generate a Heatmap: Use the ComplexHeatmap package in R to create a heatmap of the count data
   #  for the selected genes.
# iii)  Add an Annotation Bar: Include an annotation bar reflecting your chosen covariate for further

library(ComplexHeatmap)
```

```
## Loading required package: grid
```

```
## ========================================
## ComplexHeatmap version 2.26.1
## Bioconductor page: http://bioconductor.org/packages/ComplexHeatmap/
## Github page: https://github.com/jokergoo/ComplexHeatmap
## Documentation: http://jokergoo.github.io/ComplexHeatmap-reference
##
## If you use it in published research, please cite either one:
## - Gu, Z. Complex Heatmap Visualization. iMeta 2022.
## - Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional
##     genomic data. Bioinformatics 2016.
##
##
## The new InteractiveComplexHeatmap package can directly export static
## complex heatmaps into an interactive Shiny app with zero effort. Have a try!
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(ComplexHeatmap))
## ========================================
```

```r
library(circlize)
```

```
## Warning: package 'circlize' was built under R version 4.5.3
```

```
## ========================================
## circlize version 0.4.18
## CRAN page: https://cran.r-project.org/package=circlize
## Github page: https://github.com/jokergoo/circlize
## Documentation: https://jokergoo.github.io/circlize_book/book/
##
## If you use it in published research, please cite:
## Gu, Z. circlize implements and enhances circular visualization
##   in R. Bioinformatics 2014.
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(circlize))
## ========================================
```

```r
#count.df <- read.csv(file = "data/counts.csv", header = TRUE)
#meta_data.df <- read.csv(file = "data/meta_data.csv", header = TRUE)

count.df.subset <- count.df[1:10,1:11]
count.df.subset <- count.df.subset[-1]

meta_data.df.correct <- meta_data.df %>%
  mutate(barcode = str_replace_all(barcode, "-", "."))
```

```r
meta_data.df.correct <- meta_data.df.correct[1:10,]
```

```r
## Heatmap annotations
column_ha <- HeatmapAnnotation(
  Disease_Status = meta_data.df.correct$ajcc_pathologic_stage,
  Laterality = meta_data.df.correct$laterality,
  col = list(
    Disease_Status = c("Stage I" = "red", "Stage IA" = "blue", "Stage IIA" = "green",
                       "Stage IIB" = "yellow", "Stage IIIA" = "brown", "Stage IIIC" = "gray"),
    Laterality = c(Left = "green", Right = "purple")
  )
)
```

```r
# Create heatmap
Heatmap(
  count.df.subset,
  name = "Counts",
  top_annotation = column_ha,
  show_row_names = TRUE,
  show_column_names = TRUE,
  cluster_columns = TRUE,
  cluster_rows = TRUE,
  column_title = "Samples",
  row_title = "Genes",
  heatmap_legend_param = list(title = "Count")
)
```

